



COMPUTER NETWORK • PRACTICAL FILE

STUDENT NAME

Deven

UID

24BCF10060

BRANCH

AIT-CSE

SECTION / GROUP

24AIT_KRG-G2

SEMESTER

5

SUBJECT

Computer Network

SUBJECT CODE

24CSH-337

UNIVERSITY

Chandigarh University

9

Implement Connection using UDP

UDP • CONNECTIONLESS

Objective

- ✓ Understand the basic working principles and characteristics of UDP (User Datagram Protocol).
- ✓ Implement UDP client-server communication using Python socket programming.
- ✓ Establish communication between **server.py** and **client.py** using UDP sockets.
- ✓ Observe how datagrams are sent and received without a dedicated connection between client and server.

Theory

UDP (User Datagram Protocol) is a connectionless transport-layer protocol used for communication between devices on a network. Unlike TCP, it does not set up a connection before data is sent and offers no guarantee of delivery, ordering, or retransmission. Data instead travels as independent **datagrams**, each one carrying the destination IP address and port along with it.

In this experiment, the server creates a UDP socket and binds it to an IP address and port using **bind()**, while the client sends data straight to the server using **sendto()** (or, once "connected" for convenience, **send()**). The server reads incoming datagrams with **recvfrom()** and can reply directly to the sender. Because there is no three-way handshake or persistent session, UDP is lighter and faster than TCP – which is exactly why it suits latency-sensitive use cases such as streaming, online gaming, DNS lookups, and real-time communication.

TCP VS UDP — QUICK COMPARISON

Aspect	TCP	UDP
Connection	Connection-oriented (handshake)	Connectionless
Reliability	Guaranteed, ordered delivery	Best-effort, no guarantee
Speed	Slower (overhead of checks)	Faster, lower latency
Typical use	Web, email, file transfer	Streaming, gaming, DNS

● Practical Steps

- 1 Open the project folder in a code editor such as VS Code, PyCharm, or IntelliJ.
- 2 Create two Python files named **server.py** and **client.py**.
- 3 Paste the server-side code into server.py and the client-side code into client.py.
- 4 Save both files.
- 5 Open two terminals (or a split terminal).
- 6 In Terminal 1, start the server with **python server.py**.
- 7 In Terminal 2, start the client with **python client.py**.
- 8 Exchange a few datagrams between the client and server CLIs, then type **Over** on the client side to end the session.

● Code

```
server.py

import socket

server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server_socket.bind(("127.0.0.1", 5000))
print("UDP Server started")
print("Waiting for messages...")

while True:
    message, client_address = server_socket.recvfrom(1024)
    message = message.decode()
    print("Client:", message)
    if message == "Over":
        break
    reply = input("Enter reply: ")
    server_socket.sendto(reply.encode(), client_address)

server_socket.close()
print("Server closed")
```

```
client.py

import socket

client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
client_socket.connect(("127.0.0.1", 5000))
print("UDP Client started")

while True:
    message = input("Enter message: ")
    client_socket.send(message.encode())  ## encode(): string -> bytes
    if message == "Over":
        break
    reply = client_socket.recv(1024).decode()
    print("Server:", reply)

client_socket.close()
print("Client closed")
```

● Result

```
client.py – Terminal 2

PS D:\Deven\CN_Lab\UDP_Experiment> python client.py
UDP Client started
Enter message: Hello server, this is a UDP test from Deven
Server: Datagram received, no handshake needed here
Enter message: Exactly, that is the whole point of UDP
Server: Right – fast and lightweight, but no delivery guarantee
Enter message: Good for real-time stuff like gaming and DNS
then
Server: Exactly that
Enter message: Over
Client closed
```

```
server.py – Terminal 1

PS D:\Deven\CN_Lab\UDP_Experiment> python server.py
UDP Server started
Waiting for messages...
Client: Hello server, this is a UDP test from Deven
Enter reply: Datagram received, no handshake needed here
Client: Exactly, that is the whole point of UDP
Enter reply: Right – fast and lightweight, but no delivery
guarantee
Client: Good for real-time stuff like gaming and DNS then
Enter reply: Exactly that
Client: Over
Server closed
```

● Learning Outcome

- ★ Understood the working and characteristics of UDP and how it differs from connection-oriented protocols such as TCP.
- ★ Built and ran a UDP client-server application in Python for sending and receiving datagrams.
- ★ Compared UDP against TCP in terms of reliability, overhead, and typical real-world use cases.